Hands-On SQL Projects for Database Learning (Beginner-level)

Project Title

Student Database Management System

Project Overview

The **Student Database Management System** is a database project designed to manage student information in an educational institution.
Many schools and training centers still rely on manual or poorly organized systems to store student records, which leads to data duplication, inconsistency, and difficulty in accessing information.

This project focuses on designing and implementing a **relational database** using **MySQL** to store, manage, and retrieve student data efficiently.

Tools Used

- MySQL
- SQL (Structured Query Language)
- MySQL Workbench
- GitHub
- ER Diagram Tool (Draw.io or Lucidchart)

Problem Statement

Educational institutions need a reliable way to store and manage student information such as personal details, enrollment data, and academic programs.
Without a structured database system, institutions face challenges such as:

- Duplicate student records
- Difficulty updating student information
- Poor data organization
- Slow retrieval of student details

The goal of this project is to design a well-structured database that solves these problems using proper database design principles.

Project Questions / Tasks

1. Database Model

Explain the **relational database model** and why it is suitable for managing student data.
Compare it with other database models such as hierarchical and network models.

2. Entities, Relationships, and Attributes

Identify the main entities involved in a Student Database Management System.
Define their attributes and relationships.
Draw an **Entity Relationship (ER) diagram** showing primary keys and relationships.

3. Normalization

Initially, all student information is stored in a single table that includes student details, department information, and enrollment year.

- Explain the problems with this design.
- Explain the importance of normalization.
- Normalize the database up to **Third Normal Form (3NF)** and show all steps.

4. Dataset Description (Required Tables)

Database Name

student_db

Table 1: Students

Column Name	Data Type	Description
student_id	INT (PK)	Unique student identifier
first_name	VARCHAR(50)	Student first name
last_name	VARCHAR(50)	Student last name
gender	VARCHAR(10)	Student gender
date_of_birth	DATE	Date of birth
enrollment_year	YEAR	Year of enrollment
department_id	INT (FK)	References Departments

Table 2: Departments

Column Name	Data Type	Description
department_id	INT (PK)	Department identifier
department_name	VARCHAR(100)	Name of department

Sample Data

Departments Table

department_id	department_name
1	Information Technology
2	Business Administration
3	Electrical Engineering

Students Table

student_id	first_name	last_name	gender	date_of_birth	enrollment_year	department_id
101	Alice	Uwimana	Female	2003-05-12	2023	1
102	Jean	Ndayishimiye	Male	2002-11-08	2022	2
103	Grace	Mukamana	Female	2004-02-20	2023	1

5. MySQL Tasks

Write SQL queries to:

1. Create the database student_db.
2. Create the Departments table.
3. Create the Students table with appropriate primary and foreign keys.
4. Insert the provided sample data into both tables.
5. Retrieve all students from a specific department.
6. Update the department of a student using student_id.
7. Delete a student record using student_id.

Project Usefulness

This project helps institutions store and manage student records in a structured way.
It improves data consistency, reduces redundancy, and allows quick access to student information.
The project also prepares students for real-world database applications.

Skills You Will Gain

After completing this project, students will gain:

- Strong understanding of relational databases
- Ability to design ER diagrams
- Knowledge of database normalization (up to 3NF)
- Practical MySQL and SQL querying skills
- Experience organizing and documenting projects on GitHub
- Confidence in solving real-life data management problems

Project Title

Library Book Management System

Project Overview

The **Library Book Management System** is designed to manage books, library members, and book borrowing activities in a school or public library.
Many libraries still face challenges such as poor record keeping, missing books, and difficulty tracking borrowed items due to manual systems.

This project focuses on designing and implementing a **relational database using MySQL** to efficiently manage library operations.

Tools Used

- MySQL
- SQL (Structured Query Language)
- MySQL Workbench
- GitHub
- ER Diagram Tool (Draw.io / Lucidchart)

Problem Statement

Libraries need an organized way to manage books and track which members have borrowed them.
Without a proper database system, libraries experience:

- Loss of books
- Difficulty tracking borrowed and returned books
- Duplicate book and member records
- Slow access to information

This project aims to solve these problems by building a structured and normalized database system.

Project Questions / Tasks

1. Database Model

Explain why the **relational database model** is suitable for a library management system.
Discuss how tables, primary keys, and foreign keys help maintain data integrity.

2. Entities, Relationships, and Attributes

Identify the main entities in the library system.
Define their attributes and relationships.
Draw an **ER diagram** showing all relationships clearly.

3. Normalization

Initially, all library data (books, members, borrow details) is stored in a single table.

- Explain the problems with this design
- Explain why normalization is necessary
- Normalize the database up to **Third Normal Form (3NF)**

Dataset Description (Required Tables)

Database Name

library_db

Table 1: Books

Column Name	Data Type	Description
book_id	INT (PK)	Unique book identifier
title	VARCHAR(150)	Book title
author	VARCHAR(100)	Book author
category	VARCHAR(50)	Book category
published_year	YEAR	Year published
copies_available	INT	Number of copies

Table 2: Members

Column Name	Data Type	Description
member_id	INT (PK)	Library member ID
first_name	VARCHAR(50)	First name
last_name	VARCHAR(50)	Last name
gender	VARCHAR(10)	Gender
phone	VARCHAR(15)	Contact number
membership_date	DATE	Registration date

Table 3: Borrowings

Column Name	Data Type	Description
borrow_id	INT (PK)	Borrow record ID
member_id	INT (FK)	References Members
book_id	INT (FK)	References Books
borrow_date	DATE	Date borrowed
return_date	DATE	Date returned

Sample Data

Books

book_id	title	author	category	published_year	copies_available
1	Database Systems	Thomas Connolly	Technology	2019	5
2	Clean Code	Robert Martin	Programming	2018	3
3	Introduction to Economics	Paul Samuelson	Economics	2016	4
4	Discrete Mathematics	Kenneth Rosen	Mathematics	2020	2

Members

member_id	first_name	last_name	gender	phone	membership_date
101	Alice	Uwimana	Female	0788123456	2023-01-10
102	Jean	Ndayishimiye	Male	0788345678	2023-02-15
103	Grace	Mukamana	Female	0788567890	2023-03-05

Borrowings

borrow_id	member_id	book_id	borrow_date	return_date
1	101	1	2024-01-10	2024-01-20
2	102	2	2024-01-15	NULL
3	103	3	2024-01-18	2024-01-28

MySQL Tasks / SQL Statements

Students must write SQL queries to:

1. Create the database library_db.
2. Create the Books, Members, and Borrowings tables with proper keys.
3. Insert all sample data.
4. Retrieve all books available in the library.
5. List all books borrowed by a specific member.
6. Show members who currently borrowed books (not yet returned).
7. Update the number of available copies when a book is borrowed.
8. Delete a member record using member_id.
9. Use an **INNER JOIN** to display member names and book titles borrowed.

Project Usefulness

This project helps libraries manage books and borrowing activities efficiently.
It ensures accurate tracking of library resources and improves service delivery.
The project also exposes students to real-world database problems.

Skills You Will Gain

After completing this project, students will gain:

- Database design and modeling skills
- Ability to work with multiple related tables
- Strong understanding of primary and foreign keys
- Practical experience using SQL JOINs
- Data normalization skills
- Experience documenting projects professionally on GitHub

Project Title

Employee Payroll Tracking System

Project Overview

The **Employee Payroll Tracking System** is designed to manage employee information, salary structures, and monthly payroll records within an organization.
Many companies struggle with payroll errors, delayed payments, and poor record management when using manual or spreadsheet-based systems.

This project focuses on designing and implementing a **relational database using MySQL** to accurately calculate, store, and track employee payroll data.

Tools Used

- MySQL
- SQL (Structured Query Language)
- MySQL Workbench
- GitHub
- ER Diagram Tool (Draw.io / Lucidchart)

Problem Statement

Organizations must ensure that employees are paid accurately and on time.
Without a structured payroll database, companies face:

- Salary miscalculations
- Inconsistent payroll records

- Difficulty tracking deductions and allowances
- Limited payroll reporting

This project aims to create a reliable payroll tracking system that eliminates these challenges.

Project Questions / Tasks

1. Database Model

Explain why a **relational database** is suitable for an employee payroll system.
Discuss how tables, relationships, and constraints help ensure payroll accuracy.

2. Entities, Relationships, and Attributes

Identify the main entities in the payroll system.
Define their attributes and relationships.
Create an **ER diagram** representing the database structure.

3. Normalization

Initially, all payroll data (employee details, salary, deductions, payments) is stored in one table.

- Explain the problems with this approach
- Explain why normalization is required
- Normalize the database up to **Third Normal Form (3NF)**

Dataset Description (Required Tables)

Database Name

employee_payroll_db

Table 1: Departments

Column Name	Data Type	Description
department_id	INT (PK)	Department identifier
department_name	VARCHAR(100)	Department name

Table 2: Employees

Column Name	Data Type	Description
employee_id	INT (PK)	Employee ID
first_name	VARCHAR(50)	First name
last_name	VARCHAR(50)	Last name
gender	VARCHAR(10)	Gender
hire_date	DATE	Employment start date
department_id	INT (FK)	References Departments
base_salary	DECIMAL(10,2)	Monthly base salary

Table 3: Payroll

Column Name	Data Type	Description
payroll_id	INT (PK)	Payroll record ID
employee_id	INT (FK)	References Employees
pay_month	VARCHAR(20)	Payroll month
allowances	DECIMAL(10,2)	Allowances
deductions	DECIMAL(10,2)	Deductions
net_salary	DECIMAL(10,2)	Final salary

Sample Data

Departments

department_id	department_name
1	Human Resources
2	Finance
3	Information Technology

Employees

employee_id	first_name	last_name	gender	hire_date	department_id	base_salary
201	Alice	Uwimana	Female	2021-03-01	1	450000
202	Jean	Ndayishimiye	Male	2020-06-15	2	600000
203	Grace	Mukamana	Female	2022-01-10	3	550000

Payroll

payroll_id	employee_id	pay_month	allowances	deductions	net_salary
1	201	January 2024	50000	20000	480000
2	202	January 2024	80000	30000	650000
3	203	January 2024	60000	25000	585000

MySQL Tasks / SQL Statements

Students must write SQL queries to:

1. Create the database employee_payroll_db.
2. Create all tables with proper primary and foreign keys.
3. Insert the sample data into each table.
4. Retrieve all employees from a specific department.
5. Calculate net salary using base salary, allowances, and deductions.
6. Display payroll details for a specific month.
7. Update an employee's base salary.
8. Delete a payroll record using payroll_id.
9. Use **JOINs** to display employee names, department names, and net salaries.

Project Usefulness

This project helps organizations manage payroll efficiently and accurately.
It improves salary calculation transparency and ensures proper payroll record management.
Students also gain exposure to real-world business database systems.

Skills You Will Gain

After completing this project, students will gain:

- Payroll system database design skills
- Experience working with financial data
- Strong understanding of relational database relationships
- Advanced SQL querying and JOIN operations
- Data normalization and integrity enforcement
- Professional project documentation skills for GitHub

Project Title

Inventory Stock Control System

Project Overview

The **Inventory Stock Control** System is a database project designed to help businesses manage products, suppliers, and stock movements efficiently.
The system tracks inventory levels, records stock coming in and going out, and helps prevent issues such as stock shortages, overstocking, and data inconsistency.

This project uses a **relational database** and **MySQL** to model real-world inventory operations in a structured and reliable way.

Tools Used

- MySQL
- SQL (Structured Query Language)
- MySQL Workbench
- ER Diagram Tool (Draw.io / Lucidchart)
- GitHub (project documentation and version control)

Problem Statement

Many businesses manage inventory manually or using unstructured spreadsheets. This leads to:

- Inaccurate stock quantities
- Difficulty tracking suppliers and products
- Poor visibility of stock movement
- Losses due to overstocking or stock shortages

There is a need for a centralized database system that can accurately store inventory data and support efficient stock management.

Full Project Questions / Tasks

1. Database Design

- Explain why a **relational database model** is suitable for an inventory stock control system.
- Identify the main entities and their relationships.
- Design an **Entity Relationship (ER) diagram** showing all tables, primary keys, and foreign keys.

2. Normalization

- Assume all inventory data is stored in one table.
- Explain the problems with this design.
- Normalize the database up to **Third Normal Form (3NF)** and explain each step.

3. SQL Implementation

Using **MySQL**, write SQL queries to:

1. Create the database.
2. Create all required tables with primary and foreign keys.
3. Insert the provided sample data.
4. Retrieve all products with their category and supplier information.
5. Display products with low stock levels.
6. Record stock transactions (IN and OUT).
7. Update product quantities after transactions.
8. Generate an inventory report using JOINs.

Dataset Description

Database Name

inventory_db

Table 1: Categories

Column Name	Data Type	Description
category_id	INT (PK)	Unique category ID
category_name	VARCHAR(100)	Category name

Sample Records

category_id	category_name
1	Electronics
2	Stationery
3	Groceries

Table 2: Suppliers

Column Name	Data Type	Description
supplier_id	INT (PK)	Supplier ID
supplier_name	VARCHAR(150)	Supplier name
phone	VARCHAR(15)	Contact number
location	VARCHAR(100)	Supplier location

Sample Records

supplier_id	supplier_name	phone	location
1	TechWorld Ltd	0788111222	Kigali
2	OfficePlus	0788333444	Huye
3	FreshFoods Ltd	0788555666	Muhanga

Table 3: Products

Column Name	Data Type	Description
product_id	INT (PK)	Product ID
product_name	VARCHAR(150)	Product name
category_id	INT (FK)	References Categories
supplier_id	INT (FK)	References Suppliers
unit_price	DECIMAL(10,2)	Price per unit
quantity_in_stock	INT	Current stock

Sample Records

product_id	product_name	category_id	supplier_id	unit_price	quantity_in_stock
101	Laptop	1	1	850000	15
102	Printer Paper	2	2	12000	200
103	Rice 25kg	3	3	38000	50

Table 4: Stock_Transactions

Column Name	Data Type	Description
transaction_id	INT (PK)	Transaction ID
product_id	INT (FK)	References Products
transaction_type	VARCHAR(10)	IN or OUT
quantity	INT	Quantity moved
transaction_date	DATE	Date of transaction

Sample Records

transaction_id	product_id	transaction_type	quantity	transaction_date
1	101	IN	5	2024-01-05
2	102	OUT	20	2024-01-08
3	103	IN	10	2024-01-12

Project Usefulness

This project helps businesses maintain accurate inventory records and track stock movement effectively.
It supports better purchasing decisions, reduces losses, and improves operational efficiency.
For students, it provides real-world experience in designing and managing business databases.

Skills Students Will Gain

After completing this project, students will gain:

- Relational database design skills
- Ability to create ER diagrams
- Knowledge of database normalization (up to 3NF)
- Strong SQL skills (CREATE, INSERT, SELECT, UPDATE, DELETE, JOIN)
- Understanding of inventory management concepts
- Experience documenting projects professionally on GitHub

Project Title

Hospital Patient Records Management System

Project Overview

The **Hospital Patient Records Management System** is a database project designed to manage patient information, medical history, and appointments in a hospital environment. Many hospitals still rely on paper-based or poorly organized systems, which leads to errors, data loss, and slow access to patient records.

This project focuses on designing and implementing a **relational database using MySQL** to securely store and manage patient records in an efficient and structured way.

Tools Used

- MySQL
- SQL (Structured Query Language)
- MySQL Workbench
- ER Diagram Tool (Draw.io / Lucidchart)
- GitHub (project documentation and version control)

Problem Statement

Hospitals handle large volumes of patient data every day. When records are managed manually, the hospital may experience:

- Loss or duplication of patient records
- Difficulty tracking patient medical history
- Slow retrieval of patient information
- Inconsistent and inaccurate data

There is a need for a centralized electronic database system to manage patient records accurately and securely.

Full Project Questions / Tasks

1. Database Design

- Explain why a **relational database model** is suitable for managing hospital patient records.
- Identify the main entities and their relationships.
- Design an **Entity Relationship (ER) diagram** showing tables, primary keys, and foreign keys.

2. Normalization

- Assume all patient data (personal details, visits, diagnoses) is stored in one table.

- Explain the problems with this design.
- Normalize the database up to **Third Normal Form (3NF)** and explain each step

3. SQL Implementation

Using **MySQL**, write SQL queries to:

1. Create the database.
2. Create all required tables with proper constraints.
3. Insert the provided sample data.
4. Retrieve patient medical history.
5. List patients who visited the hospital on a specific date.
6. Update patient information.
7. Delete a patient record.
8. Use JOINs to generate patient reports.

Dataset Description

Database Name

hospital_db

Table 1: Patients

Column Name	Data Type	Description
patient_id	INT (PK)	Unique patient ID
first_name	VARCHAR(50)	First name
last_name	VARCHAR(50)	Last name
gender	VARCHAR(10)	Gender
date_of_birth	DATE	Date of birth
phone	VARCHAR(15)	Contact number

Sample Records

patient_id	first_name	last_name	gender	date_of_birth	phone
1	Alice	Uwimana	Female	2000-05-10	0788001111
2	Jean	Ndayishimiye	Male	1998-09-15	0788002222
3	Grace	Mukamana	Female	2002-01-20	0788003333

Table 2: Doctors

Column Name	Data Type	Description
doctor_id	INT (PK)	Doctor ID
doctor_name	VARCHAR(100)	Doctor name
specialization	VARCHAR(100)	Medical specialty

Sample Records

doctor_id	doctor_name	specialization
1	Dr. Mugisha	General Medicine
2	Dr. Uwase	Pediatrics
3	Dr. Habimana	Surgery

Table 3: Appointments

Column Name	Data Type	Description
appointment_id	INT (PK)	Appointment ID
patient_id	INT (FK)	References Patients
doctor_id	INT (FK)	References Doctors
appointment_date	DATE	Date of visit
diagnosis	VARCHAR(200)	Diagnosis notes

Sample Records

appointment_id	patient_id	doctor_id	appointment_date	diagnosis
1	1	1	2024-01-10	Malaria
2	2	2	2024-01-12	Flu
3	3	1	2024-01-15	Stomach pain

Project Usefulness

This project helps hospitals maintain organized and secure patient records.
It improves access to medical history, reduces errors, and supports better healthcare delivery.
For students, it provides practical experience working with real hospital data scenarios.

Skills Students Will Gain

After completing this project, students will gain:

- Hospital database system design skills
- Experience modeling real-world healthcare data
- Ability to create ER diagrams
- Knowledge of database normalization (up to 3NF)
- Strong SQL skills including JOINs and filtering
- Professional GitHub documentation skills

Project Title

Hospital Patient Records Management System

Project Overview

The **Hospital Patient Records Management System** is a database project designed to manage patient information, medical history, and appointments in a hospital environment. Many hospitals still rely on paper-based or poorly organized systems, which leads to errors, data loss, and slow access to patient records.

This project focuses on designing and implementing a **relational database using MySQL** to securely store and manage patient records in an efficient and structured way.

Tools Used

- MySQL
- SQL (Structured Query Language)
- MySQL Workbench
- ER Diagram Tool (Draw.io / Lucidchart)
- GitHub (project documentation and version control)

Problem Statement

Hospitals handle large volumes of patient data every day. When records are managed manually, the hospital may experience:

- Loss or duplication of patient records
- Difficulty tracking patient medical history
- Slow retrieval of patient information
- Inconsistent and inaccurate data

There is a need for a centralized electronic database system to manage patient records accurately and securely.

Full Project Questions / Tasks

1. Database Design

- Explain why a **relational database model** is suitable for managing hospital patient records.
- Identify the main entities and their relationships.
- Design an **Entity Relationship (ER) diagram** showing tables, primary keys, and foreign keys.

2. Normalization

- Assume all patient data (personal details, visits, diagnoses) is stored in one table.
- Explain the problems with this design.
- Normalize the database up to **Third Normal Form (3NF)** and explain each step.

3. SQL Implementation

Using **MySQL**, write SQL queries to:

1. Create the database.
2. Create all required tables with proper constraints.
3. Insert the provided sample data.
4. Retrieve patient medical history.
5. List patients who visited the hospital on a specific date.
6. Update patient information.
7. Delete a patient record.
8. Use JOINs to generate patient reports.

Dataset Description

Database Name

hospital_db

Table 1: Patients

Column Name	Data Type	Description
patient_id	INT (PK)	Unique patient ID
first_name	VARCHAR(50)	First name
last_name	VARCHAR(50)	Last name
gender	VARCHAR(10)	Gender
date_of_birth	DATE	Date of birth
phone	VARCHAR(15)	Contact number

Sample Records

patient_id	first_name	last_name	gender	date_of_birth	phone
1	Alice	Uwimana	Female	2000-05-10	0788001111
2	Jean	Ndayishimiye	Male	1998-09-15	0788002222
3	Grace	Mukamana	Female	2002-01-20	0788003333

Table 2: Doctors

Column Name	Data Type	Description
doctor_id	INT (PK)	Doctor ID
doctor_name	VARCHAR(100)	Doctor name
specialization	VARCHAR(100)	Medical specialty

Sample Records

doctor_id	doctor_name	specialization
1	Dr. Mugisha	General Medicine
2	Dr. Uwase	Pediatrics
3	Dr. Habimana	Surgery

Table 3: Appointments

Column Name	Data Type	Description
appointment_id	INT (PK)	Appointment ID
patient_id	INT (FK)	References Patients
doctor_id	INT (FK)	References Doctors
appointment_date	DATE	Date of visit
diagnosis	VARCHAR(200)	Diagnosis notes

Sample Records

appointment_id	patient_id	doctor_id	appointment_date	diagnosis
1	1	1	2024-01-10	Malaria
2	2	2	2024-01-12	Flu
3	3	1	2024-01-15	Stomach pain

Project Usefulness

This project helps hospitals maintain organized and secure patient records.
It improves access to medical history, reduces errors, and supports better healthcare delivery.
For students, it provides practical experience working with real hospital data scenarios.

Skills Students Will Gain

After completing this project, students will gain:

- Hospital database system design skills
- Experience modeling real-world healthcare data
- Ability to create ER diagrams
- Knowledge of database normalization (up to 3NF)
- Strong SQL skills including JOINs and filtering
- Professional GitHub documentation skills

Project Title

Restaurant Reservation System

Project Overview

The **Restaurant Reservation System** is a database project designed to manage customer information, table reservations, and dining schedules in a restaurant.
Many restaurants rely on phone calls or manual reservation books, which can lead to double bookings, lost reservations, and poor customer experience.

This project focuses on designing and implementing a **relational database using MySQL** to efficiently manage reservations and improve restaurant operations.

Tools Used

- MySQL
- SQL (Structured Query Language)
- MySQL Workbench
- ER Diagram Tool (Draw.io / Lucidchart)
- GitHub (project documentation and version control)

Problem Statement

Restaurants need an organized system to manage reservations and table availability.
Without a proper database system, restaurants face:

- Double booking of tables
- Difficulty tracking customer reservations
- Poor management of table capacity
- Inefficient service planning

This project aims to create a structured database system that ensures accurate reservation management.

Full Project Questions / Tasks

1. Database Design

- Explain why a **relational database model** is suitable for a restaurant reservation system.
- Identify the main entities and their relationships.
- Design an **Entity Relationship (ER) diagram** showing tables, primary keys, and foreign keys.

2. Normalization

- Assume all reservation data (customer details, tables, reservation times) is stored in one table.
- Explain the problems with this design.
- Normalize the database up to **Third Normal Form (3NF)**.

3. SQL Implementation

Using **MySQL**, write SQL queries to:

1. Create the database.
2. Create all required tables with proper constraints.
3. Insert the provided sample data.
4. Retrieve all reservations for a specific date.
5. Check available tables for a given time slot.
6. Update reservation details.
7. Cancel a reservation.
8. Use JOINs to generate reservation reports.

Dataset Description

Database Name

restaurant_db

Table 1: Customers

Column Name	Data Type	Description
customer_id	INT (PK)	Customer ID
first_name	VARCHAR(50)	First name
last_name	VARCHAR(50)	Last name
phone	VARCHAR(15)	Contact number

Sample Records

customer_id	first_name	last_name	phone
1	Alice	Uwimana	0789001111
2	Jean	Ndayishimiye	0789002222
3	Grace	Mukamana	0789003333

Table 2: Tables

Column Name	Data Type	Description
table_id	INT (PK)	Table ID
table_number	INT	Table number
capacity	INT	Number of seats

Sample Records

table_id	table_number	capacity
1	1	2
2	2	4
3	3	6

Table 3: Reservations

Column Name	Data Type	Description
reservation_id	INT (PK)	Reservation ID
customer_id	INT (FK)	References Customers
table_id	INT (FK)	References Tables
reservation_date	DATE	Reservation date
reservation_time	TIME	Reservation time

Sample Records

reservation_id	customer_id	table_id	reservation_date	reservation_time
1	1	2	2024-02-10	18:00
2	2	3	2024-02-10	19:00
3	3	1	2024-02-11	17:30

Project Usefulness

This project helps restaurants manage reservations efficiently and avoid booking conflicts. It improves customer satisfaction and supports better planning of restaurant resources. For students, it provides practical experience with real-world database design.

Skills Students Will Gain

After completing this project, students will gain:

- Database modeling for service-based systems
- Experience designing reservation databases
- Strong understanding of relational databases
- SQL skills including JOINs, filtering, and date/time queries
- Database normalization skills
- Professional GitHub documentation skills